

Modelo predictivo de GrillosIA

Documento técnico · Actividad 3.2 Universidad de los Llanos · Convocatoria Minciencias 963 de 2025 · Contrato 207-2025

1. Qué tiene que responder el modelo

Un productor del Piedemonte Llanero cría grillos y quiere saber, antes de empezar el lote, con cuál de las comidas en estudio va a obtener una harina más cercana a lo que necesita su animal.

La pregunta se traduce así:

Dadas unas condiciones de cría, ¿cuánta proteína y cuánto lípido va a tener la harina, y cuántos grillos van a llegar vivos a la cosecha?

Entra (seis variables):

Variable	Tipo	Unidad
tipo_dieta	categorica	código D1, D2, D3
alimento_g_dia	numérica	g/día
temperatura	numérica	°C
humedad_ambiental	numérica	%
especie	categorica	familia
tiempo_desarrollo	numérica	días

Sale:

Variable	Unidad	Papel
proteina_harina	% materia seca	objetivo principal
lipidos_harina	% materia seca	objetivo principal
tasa_supervivencia	%	filtro de viabilidad

La supervivencia se predice aparte porque responde otra pregunta. No es "qué harina sale" sino "sale algo". Una comida con proteína alta y supervivencia baja no le sirve al productor, y hace falta ese número para descartarla antes de comparar.

2. La restricción que manda: cuántos datos hay

Esta es la decisión de diseño más importante y no es técnica, es aritmética.

Al cierre de este documento la plantilla de recolección tiene **siete lotes** registrados y **ninguno** con análisis bromatológico terminado: los grillos siguen creciendo y la proteína de la harina se mide sobre grillo cosechado. El proyecto contempla los tres códigos de dieta en estudio, con repeticiones, lo que sitúa el conjunto final en el orden de **decenas de filas**, no de miles.

El trabajo de referencia más cercano —Vargas-Serna et al. (2025), predicción de rendimiento proteico en larvas— trabajó con 105 puntos extraídos de 28 artículos. Ese es el orden de magnitud del problema.

Con decenas de filas y seis variables, la elección de modelo no se decide por capacidad de representación. Se decide por **cuánto se puede confiar en lo que diga el modelo cuando aprendió de tan poco**.

3. Por qué Random Forest

3.1 Lo que se comparó

Familia	Por qué se consideró	Por qué no se eligió
Regresión lineal / Ridge	La más simple, la más interpretable, la que mejor aguanta pocos datos.	Obliga a que el efecto de la temperatura sea el mismo a 24 °C que a 34 °C. Lo que se espera aquí es una relación con óptimo: la cría rinde mejor en una franja intermedia y peor a ambos lados. Una recta no puede representar eso.
Random Forest	Captura no linealidades e interacciones sin que haya que declararlas; tolera pocos datos porque cada árbol ve una muestra distinta; da importancia de variables y margen de incertidumbre sin trabajo extra.	Elegido. Ver 3.2.
Gradient boosting (XGBoost, LightGBM)	Suele superar a Random Forest en tablas.	La ventaja aparece con miles de filas. Con decenas, el boosting sobreajusta con facilidad y depende de ajustar varios hiperparámetros —tasa de aprendizaje, profundidad, número de rondas— que no se pueden calibrar honestamente sin datos de sobra para validar. Añade riesgo sin añadir respuesta.
Redes neuronales	Es lo que usó el trabajo de referencia, con $R^2 = 0,99$.	Ese R^2 se obtuvo con 105 puntos y, sobre todo, no es comparable con el que se reporta aquí: una red pequeña sobre pocas filas memoriza casi cualquier conjunto. Además no explica de dónde sale el número, y este proyecto tiene que poder decirle a un productor —y a la interventoría— por qué el sistema sugiere una comida y no otra.
SVR (máquinas de soporte vectorial)	Buena reputación con pocos datos.	No entrega naturalmente ni importancia de variables ni margen de error, que son las dos cosas que aquí hacen falta. Exigiría montarlas aparte.
k-vecinos más cercanos	Trivial de explicar: "le digo lo que le pasó al lote más parecido".	Con decenas de filas repartidas en tres dietas, "el más parecido" puede estar muy lejos. Y no distingue entre una zona bien cubierta por los ensayos y una vacía.

3.2 Las cuatro razones de la elección

Aguanta pocos datos sin desbaratarse. Cada árbol se entrena sobre una muestra distinta y con un subconjunto distinto de variables. Promediar 300 árboles así reduce la varianza, que es exactamente el problema cuando hay pocas filas. Un solo árbol sobre 40 filas es inservible; un bosque de 300 no.

No hay que decirle la forma de la relación. No se sabe de antemano si la proteína cae de forma suave o abrupta al subir la temperatura, ni si el efecto de la humedad depende de la dieta. Un bosque encuentra esos cortes por su cuenta. Una regresión habría exigido escribir la fórmula a mano, y no hay evidencia propia todavía para escribirla.

Dice cuánto pesó cada variable. La importancia de variables sale del entrenamiento sin trabajo adicional, y es un resultado del proyecto por sí misma: saber que la dieta pesa más que la temperatura, o al revés, orienta el diseño de los ensayos siguientes.

Dice cuánto duda. Los 300 árboles votan. Cuando coinciden, el margen es estrecho; cuando discrepan, es ancho. Ese margen es el $\pm$ que la aplicación muestra junto al número, y es lo que impide presentar como firme una predicción sobre condiciones que los ensayos casi no cubrieron.

3.3 Lo que se pierde y se acepta

- **No extrapola.** Un bosque nunca predice fuera del rango de valores que vio. Si todos los ensayos van de 26 °C a 30 °C, para 34 °C devolverá algo parecido a lo de 30 °C. Es una limitación real y por eso el sistema marca de forma explícita cuándo una consulta cae fuera del rango del estudio, en lugar de contestar como si nada.
- **No inventa combinaciones.** Solo puede hablar de las comidas que se ensayaron. Una dieta nueva exige ensayarla; el modelo no la deduce.
- **Es más pesado que una fórmula.** El artefacto entrenado ocupa unos pocos megabytes, frente a los seis coeficientes de una regresión. Irrelevante al desplegarlo en un servidor pequeño.

4. Cómo está construido

```
ColumnTransformer
├─ OneHotEncoder      → tipo_dieta, especie
└─ StandardScaler    → alimento_g_dia, temperatura,
                       humedad_ambiental, tiempo_desarrollo
    |
    ▼
MultiOutputRegressor(RandomForestRegressor)
├─ modelo principal   → proteina_harina, lipidos_harina
└─ modelo de viabilidad → tasa_supervivencia
```

Decisiones de implementación y su motivo:

- **Todo dentro de un Pipeline** . La codificación y el escalado se guardan junto al modelo. Así es imposible que la transformación aplicada al predecir se desvíe de la que se aplicó al entrenar, que es la forma más común de que un modelo funcione en pruebas y falle en producción.
- **OneHotEncoder(handle_unknown="ignore")** . Si mañana llega una consulta con un código de dieta que el modelo no vio, no revienta.
- **StandardScaler aunque un bosque no lo necesite**. No cambia el resultado. Se deja porque mantiene el preprocesamiento igual para todos los modelos y permite comparar Random Forest contra una regresión sin cambiar el pipeline cuando llegue el momento de esa comparación con datos reales.
- **MultiOutputRegressor** . Entrena un bosque por salida. Proteína y lípidos están correlacionados, pero no idénticamente determinados; separarlos permite además medir por separado qué tan bien se predice cada uno.
- **Dos modelos, no tres salidas en uno**. La viabilidad se entrena aparte porque puede haber lotes con supervivencia registrada y sin bromatología —hoy son todos—, y así esos datos no se desperdician.
- **random_state=42** . El mismo CSV produce el mismo modelo. Un resultado que no se puede repetir no se puede defender.

Código: `backend/ml/features.py` (definición de variables), `backend/ml/trainer.py` (entrenamiento), `backend/ml/predictor.py` (inferencia).

5. Cómo se mide, y qué no se acepta como medida

El R^2 sobre los datos de entrenamiento no se reporta como resultado. Un Random Forest sobre pocas filas alcanza valores altísimos sobre lo que ya vio, y ese número no dice nada sobre lo que hará con un lote nuevo. En este sistema solo se guarda para compararlo con el de validación: si la brecha entre ambos supera 0,35, el entrenamiento deja una advertencia escrita en el artefacto.

La métrica que cuenta es la validación cruzada de 5 particiones. El modelo se entrena cinco veces, cada una dejando fuera un quinto de los datos, y se evalúa sobre la parte que no vio. Se reporta:

- **R^2 por objetivo**: qué fracción de la variación explica.
- **Error absoluto medio** en puntos de porcentaje: el número interpretable. "Se equivoca en promedio 1,2 puntos de proteína" se entiende; un R^2 no.

Ambos se calculan **por separado para proteína y para lípidos**. Las funciones de puntuación de scikit-learn promedian las dos salidas en un solo número y eso taparía que una se predice mejor que la otra.

Con menos de 30 muestras el entrenamiento añade una advertencia al artefacto: por debajo de ese umbral las métricas de validación son demasiado inestables para tomarlas en serio.

6. Lo que viaja con cada predicción

Cada respuesta de la API lleva, además del número:

- `margen_proteina` / `margen_lipidos` — la dispersión entre los votos de los árboles, en puntos de porcentaje. Es el $\pm$ que se muestra en pantalla.
- `fuera_de_rango` — qué variables de la consulta caen fuera del terreno donde el modelo aprendió. Si trae algo, la predicción es una extrapolación.
- `datos_simulados`, `origen_datos`, `n_muestras`, `entrenado_en` — de dónde sale lo que se está respondiendo.

Ese último bloque no es opcional ni decorativo. Mientras el modelo esté entrenado con datos simulados, la aplicación tiene que poder decirlo en la misma pantalla donde muestra el número. Un dato que se ve bien no debe poder pasar por real.

Cuando no hay modelo entrenado, la API responde **503** y no 500: no es una avería, es que esa pieza todavía no existe. La aplicación lo entiende y deja los valores en "por confirmar" en lugar de mostrar un error.

7. Estado actual

El cableado completo está montado y probado de punta a punta: plantilla → consolidación → entrenamiento → artefacto → API → aplicación web.

Para poder probarlo antes de que lleguen los análisis bromatológicos se generó un conjunto **simulado** de 120 filas (`data/sintetico/`, con su fórmula documentada). Las cifras de la tabla siguiente son de ese conjunto y **no son un resultado del proyecto**:

	Proteína	Lípidos
R ² en validación cruzada (5 particiones)	0,780	0,609
Error absoluto medio	1,22 puntos	1,52 puntos
R ² sobre entrenamiento (solo control)	0,944	0,904

Peso de cada variable: `tipo_dieta` 62,8 % · `temperatura` 18,4 % · `tiempo_desarrollo` 8,9 % · `humedad_ambiental` 5,2 % · `alimento_g_dia` 4,6 % · `especie` 0,0 %.

Estos números sirven para una sola cosa: comprobar que el aparato funciona. La variable `alimento_g_dia` se generó al azar y **no entra en la fórmula que produjo los datos**; que el modelo le asigne un peso bajo (4,6 %) confirma que está aprendiendo la señal y no ruido. La variable `especie` pesa cero porque hoy todos los ejemplares se registran a nivel de familia: es constante, y el modelo lo detecta correctamente.

La brecha entre validación (0,780) y entrenamiento (0,944) es la esperada en un bosque, y queda por debajo del umbral que dispara la advertencia.

8. Cómo se reemplazan los datos simulados

```
# 1. Consolidar la plantilla (Excel → CSV + base de datos)
python backend/scripts/cargar_plantilla.py --excel <plantilla.xlsx>

# 2. Reentrenar, esta vez sin la bandera --simulados
python backend/scripts/entrenar.py --datos data/experimentos.csv
```

El artefacto queda marcado como `datos_simulados: false`, la API lo propaga y el aviso desaparece solo de la aplicación. No hay que tocar código.

9. Cuándo habría que reconsiderar esta elección

La decisión de usar Random Forest está atada a la cantidad de datos, así que se revisa cuando esa cantidad cambie:

- **Si el conjunto pasa de unos cientos de filas**, vale la pena comparar contra gradient boosting con validación honesta. El pipeline está preparado: cambiar el estimador final no toca nada más.
 - **Si la validación cruzada se queda baja aun con datos reales suficientes**, el problema no es el modelo sino las variables: habría que registrar algo que hoy no se registra —composición bromatológica de la dieta, por ejemplo, que hoy no existe y por eso las dietas entran como código y no como fórmula.
 - **Si se quiere predecir dietas que nunca se ensayaron**, ningún modelo de esta familia sirve. Eso exige describir la dieta por su composición, no por su código, y eso exige el análisis bromatológico de cada ingrediente.
-

Referencia

Vargas-Serna, C. L., Ochoa-Martínez, C. I., & Vélez-Pasos, C. (2025). Neural Network for AI-Driven Prediction of Larval Protein Yield: Establishing the Protein Conversion Index (PCI) for Sustainable Insect Farming. *Sustainability*, 17(2), 652. <https://doi.org/10.3390/su17020652>